

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 7

Date: 21-08-2026

Implementation of Code Optimization Techniques

Aim:

To implement the code optimization techniques of constant folding, algebraic identities, strength reduction and dead code elimination on a sequence of three address code statements.

Tools Used:

- GCC

Code:

optimizer.c

```

/*
 * Code optimization on Three Address Code (straight-line block)
 *
 * 1. Constant folding      5 * 3      -> 15    (with constant propagation)
 * 2. Algebraic identities  x+0, x*1    -> x
 * 3. Strength reduction    x**2, x*2    -> x*x, x+x
 * 4. Dead code elimination assignments whose result is never used
 *
 * Input : one instruction per line      dest = a          (copy)
 *                                           dest = a op b      (op: + - * / % ** << >>)
 *           optional last line          live: t1, t3 ...    (variables needed after the block)
 * Usage : ./optimizer < input.txt      or      ./optimizer input.txt
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <ctype.h>

#define MAXI 200          /* maximum number of instructions */
#define MAXN 32           /* maximum length of a name      */
#define MAXV 600         /* maximum size of a variable set */
#define LINE 128         /* buffer for one printed instruction */

typedef struct {
    char dest[MAXN];      /* result                        */
    char a[MAXN];         /* first operand                 */
    char b[MAXN];         /* second operand (binary only) */
    char op[3];           /* operator; "" means a plain copy */
    int dead;             /* removed by the optimizer      */
} Instr;

static Instr prog[MAXI];
static int n = 0;        /* number of instructions        */
static char liveOut[MAXV][MAXN]; /* variables live after the block */
static int nLive = 0, haveLive = 0;

/* ----- helpers ----- */

static int isnum(const char *s)
{
    if (*s == '-') s++;
    if (!*s) return 0;
    while (*s) if (!isdigit((unsigned char)*s++)) return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    return 1;
}

static int isvar(const char *s)
{
    return s[0] && !isnum(s);
}

static void show(const Instr *in, char *buf)
{
    if (in->op[0]) sprintf(buf, "%s = %s %s %s", in->dest, in->a, in->op, in->b);
    else          sprintf(buf, "%s = %s", in->dest, in->a);
}

static void make_copy(Instr *in, const char *value)
{
    char tmp[MAXN];
    strcpy(tmp, value);          /* value may point inside the instruction */
    strcpy(in->a, tmp);
    in->op[0] = in->b[0] = '\0';
}

static int power_of_two(const char *s)          /* returns k if s == 2^k, else -1 */
{
    long long v;
    int k = 0;
    if (!isnum(s)) return -1;
    v = atoll(s);
    if (v < 1 || (v & (v - 1))) return -1;
    while (v > 1) { v >>= 1; k++; }
    return k;
}

/* ----- logging */

static int round_no = 1, round_shown = 0;
static int pass_shown[5];

static void note(int pass, const char *name, const char *before, const char *after,
                const char *why)
{
    if (round_no > 1 && !round_shown) {
        printf("\n----- Round %d ----- \n", round_no);
        round_shown = 1;
    }
    if (!pass_shown[pass]) {
        printf("\n[%d] %s\n", pass, name);
        pass_shown[pass] = 1;
    }
    printf("    %-18s ==>  %-18s (%s)\n", before, after ? after : "removed", why);
}

static void no_change(int pass, const char *name)
{
    if (round_no == 1 && !pass_shown[pass]) {
        printf("\n[%d] %s\n    (no change)\n", pass, name);
        pass_shown[pass] = 1;
    }
}

/* ----- 1. constant folding + propagation */

static int evaluate(long long x, const char *op, long long y, long long *r)
{
    long long p = 1;
    if (!strcmp(op, "+")) *r = x + y;
    else if (!strcmp(op, "-")) *r = x - y;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

else if (!strcmp(op, "*")) *r = x * y;
else if (!strcmp(op, "/")) { if (y == 0) return 0; *r = x / y; }
else if (!strcmp(op, "%")) { if (y == 0) return 0; *r = x % y; }
else if (!strcmp(op, "<<")) { if (y < 0 || y > 62) return 0; *r = x << y; }
else if (!strcmp(op, ">>")) { if (y < 0 || y > 62) return 0; *r = x >> y; }
else if (!strcmp(op, "**")) { if (y < 0 || y > 62) return 0; while (y-- > 0) p *= x; *r = p; }
else return 0;
return 1;
}

static int constant_folding(void)
{
    const char *title = "Constant Folding (with constant propagation)";
    static char kname[MAXI][MAXN], kval[MAXI][MAXN]; /* variables known to be constant */
    int nk = 0, i, k, changes = 0;
    char before[LINE], after[LINE], val[MAXN];

    for (i = 0; i < n; i++) {
        Instr *in = &prog[i];
        int propagated = 0, folded = 0;
        if (in->dead) continue;
        show(in, before);

        /* constant propagation: replace a variable by its known constant */
        for (k = 0; k < nk; k++) {
            if (!kname[k][0]) continue;
            if (!strcmp(in->a, kname[k])) { strcpy(in->a, kval[k]); propagated = 1; }
            if (in->op[0] && !strcmp(in->b, kname[k])) { strcpy(in->b, kval[k]); propagated = 1; }
        }
        /* constant folding: both operands are constants */
        if (in->op[0] && isnum(in->a) && isnum(in->b)) {
            long long r;
            if (evaluate(atoll(in->a), in->op, atoll(in->b), &r)) {
                sprintf(val, "%lld", r);
                make_copy(in, val);
                folded = 1;
            }
        }
        show(in, after);
        if (strcmp(before, after)) {
            note(1, title, before, after,
                folded ? (propagated ? "constants propagated, then evaluated"
                    : "evaluated at compile time")
                    : "constant propagated");
            changes++;
        }

        /* update the table of known constants */
        for (k = 0; k < nk; k++)
            if (!strcmp(kname[k], in->dest)) kname[k][0] = '\0';
        if (!in->op[0] && isnum(in->a)) {
            for (k = 0; k < nk && kname[k][0]; k++) ;
            if (k == nk) nk++;
            strcpy(kname[k], in->dest);
            strcpy(kval[k], in->a);
        }
    }
    if (!changes) no_change(1, title);
    return changes;
}

/* ----- 2. algebraic identities ----- */

static int algebraic_identities(void)
{
    const char *title = "Algebraic Identities";
    int i, changes = 0;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char before[LINE], after[LINE], keep[MAXN];

for (i = 0; i < n; i++) {
    Instr *in = &prog[i];
    const char *op = in->op, *why = NULL;
    int a0, b0, a1, b1;
    if (in->dead || !op[0]) continue;
    show(in, before);
    a0 = !strcmp(in->a, "0"); b0 = !strcmp(in->b, "0");
    a1 = !strcmp(in->a, "1"); b1 = !strcmp(in->b, "1");
    strcpy(keep, in->a); /* value kept when x op e = x */

    if (!strcmp(op, "+") && b0) { make_copy(in, keep); why = "x + 0 = x"; }
    else if (!strcmp(op, "+") && a0) { make_copy(in, in->b); why = "0 + x = x"; }
    else if (!strcmp(op, "-") && b0) { make_copy(in, keep); why = "x - 0 = x"; }
    else if (!strcmp(op, "-") && isvar(in->a) && !strcmp(in->a, in->b))
        { make_copy(in, "0"); why = "x - x = 0"; }
    else if (!strcmp(op, "*") && b1) { make_copy(in, keep); why = "x * 1 = x"; }
    else if (!strcmp(op, "*") && a1) { make_copy(in, in->b); why = "1 * x = x"; }
    else if (!strcmp(op, "*") && (a0 || b0))
        { make_copy(in, "0"); why = "x * 0 = 0"; }
    else if (!strcmp(op, "/") && b1) { make_copy(in, keep); why = "x / 1 = x"; }
    else if (!strcmp(op, "%") && b1) { make_copy(in, "0"); why = "x % 1 = 0"; }
    else if (!strcmp(op, "**") && b1) { make_copy(in, keep); why = "x ** 1 = x"; }
    else if (!strcmp(op, "**") && b0) { make_copy(in, "1"); why = "x ** 0 = 1"; }
    else if ((!strcmp(op, "<<") || !strcmp(op, ">>")) && b0)
        { make_copy(in, keep); why = "x shifted by 0 = x"; }

    if (why) {
        show(in, after);
        note(2, title, before, after, why);
        changes++;
    }
}
if (!changes) no_change(2, title);
return changes;
}

/* ----- 3. strength reduction */

static int strength_reduction(void)
{
    const char *title = "Strength Reduction";
    int i, k, changes = 0;
    char before[LINE], after[LINE], v[MAXN], num[MAXN];

    for (i = 0; i < n; i++) {
        Instr *in = &prog[i];
        const char *why = NULL;
        if (in->dead || !in->op[0]) continue;
        show(in, before);

        if (!strcmp(in->op, "**") && !strcmp(in->b, "2") && isvar(in->a)) {
            strcpy(in->b, in->a);
            strcpy(in->op, "*");
            why = "x ** 2 = x * x";
        }
        else if (!strcmp(in->op, "**") && (isvar(in->a) != isvar(in->b))) {
            /* one operand is a variable v, the other a constant power of two */
            strcpy(v, isvar(in->a) ? in->a : in->b);
            k = power_of_two(isvar(in->a) ? in->b : in->a);
            if (k == 1) {
                strcpy(in->a, v); strcpy(in->b, v);
                strcpy(in->op, "+");
                why = "x * 2 = x + x";
            } else if (k >= 2) {
                sprintf(num, "%d", k);
            }
        }
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

        strcpy(in->a, v); strcpy(in->b, num);
        strcpy(in->op, "<<");
        why = "x * 2^k = x << k";
    }
}
if (why) {
    show(in, after);
    note(3, title, before, after, why);
    changes++;
}
}
if (!changes) no_change(3, title);
return changes;
}

/* ----- 4. dead code elimination */

static int in_set(char set[][MAXN], int cnt, const char *v)
{
    int k;
    for (k = 0; k < cnt; k++) if (!strcmp(set[k], v)) return 1;
    return 0;
}

static int dead_code_elimination(void)
{
    const char *title = "Dead Code Elimination";
    static char live[MAXV][MAXN];
    int cnt = 0, i, k, changes = 0;
    char before[LINE], why[LINE];

    /* variables needed after the block: the declared list, or every result */
    if (haveLive) {
        for (k = 0; k < nLive; k++) strcpy(live[cnt++], liveOut[k]);
    } else {
        for (i = 0; i < n; i++)
            if (!prog[i].dead && !in_set(live, cnt, prog[i].dest))
                strcpy(live[cnt++], prog[i].dest);
    }

    /* walk backwards: an assignment is useful only if its result is live */
    for (i = n - 1; i >= 0; i--) {
        Instr *in = &prog[i];
        if (in->dead) continue;
        show(in, before);
        if (!in_set(live, cnt, in->dest)) {
            in->dead = 1;
            sprintf(why, "%s is never used", in->dest);
            note(4, title, before, NULL, why);
            changes++;
            continue;
        }
        /* the definition kills dest; the operands are used by this instruction */
        for (k = 0; k < cnt; k++)
            if (!strcmp(live[k], in->dest)) { strcpy(live[k], live[--cnt]); break; }
        if (isvar(in->a) && !in_set(live, cnt, in->a))
            strcpy(live[cnt++], in->a);
        if (in->op[0] && isvar(in->b) && !in_set(live, cnt, in->b))
            strcpy(live[cnt++], in->b);
    }
    if (!changes) no_change(4, title);
    return changes;
}

/* ----- input */

static char *read_operand(char *p, char *out)

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

{
    int k = 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p == '-' && isdigit((unsigned char)p[1])) out[k++] = *p++;
    if (isdigit((unsigned char)*p))
        while (isdigit((unsigned char)*p) && k < MAXN - 1) out[k++] = *p++;
    else if (isalpha((unsigned char)*p) || *p == '_')
        while ((isalnum((unsigned char)*p) || *p == '_') && k < MAXN - 1) out[k++] = *p++;
    out[k] = '\0';
    return k ? p : NULL;
}

static int parse_instruction(char *p)
{
    Instr *in = &prog[n];
    memset(in, 0, sizeof *in);

    if (!(p = read_operand(p, in->dest)) || isnum(in->dest)) return 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p++ != '=') return 0;
    if (!(p = read_operand(p, in->a))) return 0;
    while (isspace((unsigned char)*p)) p++;
    if (*p == '\0') return 1; /* dest = a */

    if (!strncmp(p, "***", 2) || !strncmp(p, "<<", 2) || !strncmp(p, ">>", 2)) {
        strncpy(in->op, p, 2);
        p += 2;
    } else if (strchr("+-*/%", *p)) {
        in->op[0] = *p++;
    } else {
        return 0;
    }
    if (!(p = read_operand(p, in->b))) return 0;
    while (isspace((unsigned char)*p)) p++;
    return *p == '\0'; /* dest = a op b */
}

static void parse_live(char *p)
{
    char *tok = strtok(p, ", \t");
    haveLive = 1;
    while (tok && nLive < MAXV) {
        strncpy(liveOut[nLive++], tok, MAXN - 1);
        tok = strtok(NULL, ", \t");
    }
}

static void print_program(void)
{
    int i, line = 0;
    char buf[LINE];
    for (i = 0; i < n; i++) {
        if (prog[i].dead) continue;
        show(&prog[i], buf);
        printf("    %2d: %s\n", ++line, buf);
    }
}

int main(int argc, char *argv[])
{
    FILE *fp = (argc > 1) ? fopen(argv[1], "r") : stdin;
    char text[256];
    int lineno = 0, i, changes, total, left;

    if (!fp) { perror(argv[1]); return 1; }

    while (fgets(text, sizeof text, fp)) {

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    char *s = text;
    lineno++;
    text[strcspn(text, "\r\n")] = '\0';
    while (isspace((unsigned char)*s)) s++;
    if (*s == '\0' || *s == '#') continue;
    if (!strncasecmp(s, "live:", 5)) { parse_live(s + 5); continue; }
    if (n >= MAXI || !parse_instruction(s)) {
        printf("Error: cannot parse line %d: %s\n", lineno, s);
        return 1;
    }
    n++;
}
total = n;

printf("Input Three Address Code\n");
print_program();
if (haveLive) {
    printf("Live at exit :");
    for (i = 0; i < nLive; i++) printf(" %s%s", liveOut[i], i < nLive - 1 ? ", " : "");
    printf("\n");
}

/* apply the four techniques repeatedly until nothing changes any more */
for (;;) {
    round_shown = 0;
    memset(pass_shown, 0, sizeof pass_shown);
    changes = constant_folding();
    changes += algebraic_identities();
    changes += strength_reduction();
    changes += dead_code_elimination();
    if (!changes) break;
    round_no++;
}

printf("\nOptimized Three Address Code\n");
print_program();
for (i = 0, left = 0; i < n; i++) if (!prog[i].dead) left++;
printf("\nInstructions: %d -> %d\n", total, left);
return 0;
}

```

Input (input.txt)

```

t1= 5*3
t2=a+0
t3=b*1
t4=b*d
t5=d**2
t6=b*d
live: t1, t3, t4, t5

```

The last line gives the variables needed after this block (live at exit). An assignment whose result is not in that list and is not used later is dead code.

Commands Used

```

gcc optimizer.c -o optimizer
./optimizer < input.txt

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 7 % gcc optimizer.c -o optimizer
● kushaal@Kushaals-MacBook-Air Assignment 7 % ./optimizer < input.txt
Input Three Address Code
  1:  t1 = 5 * 3
  2:  t2 = a + 0
  3:  t3 = b * 1
  4:  t4 = b * d
  5:  t5 = d ** 2
  6:  t6 = b * d
Live at exit : t1, t3, t4, t5

[1] Constant Folding (with constant propagation)
    t1 = 5 * 3      ==>  t1 = 15      (evaluated at compile time)

[2] Algebraic Identities
    t2 = a + 0      ==>  t2 = a        (x + 0 = x)
    t3 = b * 1      ==>  t3 = b        (x * 1 = x)

[3] Strength Reduction
    t5 = d ** 2     ==>  t5 = d * d     (x ** 2 = x * x)

[4] Dead Code Elimination
    t6 = b * d      ==>  removed       (t6 is never used)
    t2 = a          ==>  removed       (t2 is never used)

Optimized Three Address Code
  1:  t1 = 15
  2:  t3 = b
  3:  t4 = b * d
  4:  t5 = d * d

Instructions: 6 -> 4

```

Result of optimizing the given three address code

t2 = a + 0 becomes t2 = a, and t6 = b * d repeats t4; since neither t2 nor t6 is needed after the block, dead code elimination removes both. The statement t5 = d ** 2 is reduced to t5 = d * d.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

kushaal@Kushaals-MacBook-Air Assignment 7 % ./optimizer < input2.txt
Input Three Address Code
1: a = 4 * 8
2: b = a + 2
3: c = x * 8
4: d = y ** 2
5: e = z * 2
6: f = w - w
7: g = c + 0
8: c = 7
9: h = 10 / 2
10: k = m * 0
Live at exit : b, c, d, e, f, g, h, k

[1] Constant Folding (with constant propagation)
a = 4 * 8      ==> a = 32      (evaluated at compile time)
b = a + 2      ==> b = 34      (constants propagated, then evaluated)
h = 10 / 2     ==> h = 5       (evaluated at compile time)

[2] Algebraic Identities
f = w - w      ==> f = 0       (x - x = 0)
g = c + 0      ==> g = c       (x + 0 = x)
k = m * 0      ==> k = 0       (x * 0 = 0)

[3] Strength Reduction
c = x * 8      ==> c = x << 3  (x * 2^k = x << k)
d = y ** 2     ==> d = y * y   (x ** 2 = x * x)
e = z * 2      ==> e = z + z   (x * 2 = x + x)

[4] Dead Code Elimination
a = 32         ==> removed     (a is never used)

Optimized Three Address Code
1: b = 34
2: c = x << 3
3: d = y * y
4: e = z + z
5: f = 0
6: g = c
7: c = 7
8: h = 5
9: k = 0

Instructions: 10 -> 9

```

Another test case: constant propagation, shifts and dead code

$a = 4 * 8$ is folded to 32 and propagated into $b = a + 2$, giving $b = 34$, after which a is dead. $c = x * 8$ becomes $c = x \ll 3$, and the earlier c is kept because g uses it before c is assigned again.

Learning Outcomes:

- Learned four local optimizations: constant folding (with constant propagation), algebraic identities, strength reduction and dead code elimination.
- Understood that dead code elimination depends on liveness: the block is scanned backwards and an assignment is removed when its result is not live.
- Learned that one optimization can enable another ($x * 1$ becomes a copy, which may then be dead), so the passes are repeated until nothing changes.
- Gained experience in parsing three address code in C and rewriting instructions safely, for example by not folding a division by zero.